

The Magic Mirror:

Introduction to Recursion in Python



Story Time: Vikram's Magical Mirror Palace

Vikram found a magical mirror that showed smaller mirrors inside, going on and on!
Can you imagine seeing yourself getting smaller and smaller, forever?



What is Re-cursion?

Recursion means a function calling itself — like the magical mirror showing smaller mirrors inside!

- A way to solve problems by breaking them down
- Each smaller problem looks just like the original
- Keep going until you reach the simplest answer!



Two Important Rules of Recursion!



Rule 1: Base Case

When to STOP calling the function — like a red signal saying "stop here!"



Rule 2: Recursive Case

When the function calls itself again — like a green signal saying "keep going!"

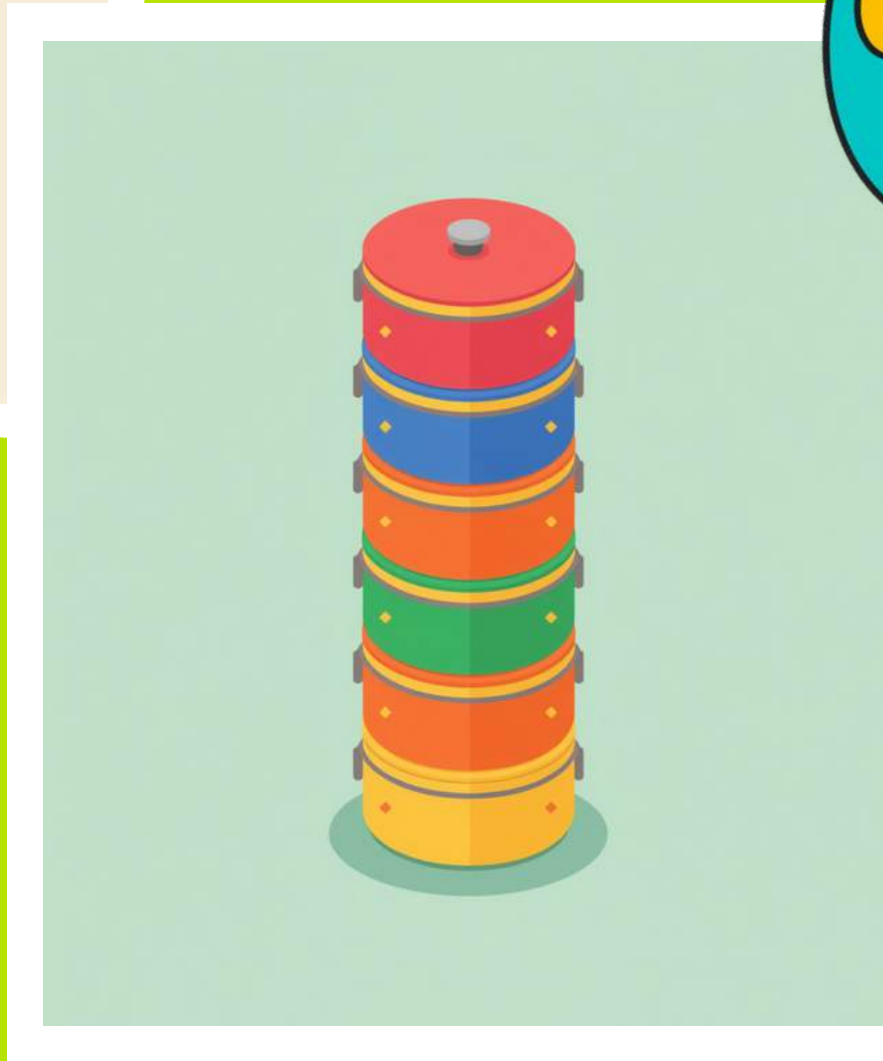


Remember!

Every recursive function needs BOTH rules to work safely and correctly.

The Call Stack: Like Steel Tiffin Boxes

- Each function call stacks up like steel tiffin boxes — one on top of another!
- Call 1 goes in first, then Call 2, then Call 3...
 - The last call finishes first and comes off the top
 - Python remembers each step until the base case is reached!





Laddu

Distribution Challenge!

Vikram needs to share laddus one by one.
Let's write a function to do that!

```
def give_laddus(n):  
    if n == 0:      # Base case  
        return  
    print("Laddu!") # Give one  
    give_laddus(n - 1) # Recursive case
```


What Happens if We Don't Stop?

If we don't have a base case, the function keeps calling itself forever!

⚠️ RecursionError: maximum recursion depth exceeded

Always remember your base case — or the mirror never stops!



Quick Quiz:

Spot the Base Case!

Which part of the laddu function is the base case?
Think and discuss!



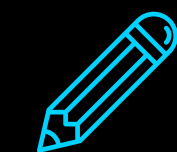
Option A

if `n == 0`: return 0
This stops the function from calling itself again!



Option B

return `n + distribute(n - 1)`
This calls the function again with a smaller number.



Option C

def `distribute(n)`:
This is just the function definition — not a case!





Debugging Together

Let's fix a small error in the laddu function to stop infinite loops. Add a base case and watch it work!

Before: missing base case causes RecursionError.

After: base case stops the function correctly!

Recap: What Did We Learn Today?



Recursion: Function Calls Itself!

A function that calls itself to solve smaller pieces of the same problem — like a mirror inside a mirror!



Two Rules: Base Case & Recursive Case

Base Case tells the function when to STOP. Recursive Case tells it to keep going and call itself again!



Call Stack: Like Stacked Tiffin Boxes!

Every function call stacks up one on top of another — just like colourful steel tiffin boxes!



Your Micro-Challenge!

Sum Numbers from 1 to N

Try writing a recursive function to add all numbers from 1 up to N.

Starter Code:

```
def sum_to_n(n):  
    if n == 1: # Base case!  
        return 1  
    return n + sum_to_n( __ )
```

Hint: What should the function call itself with?

Think: $\text{sum_to_n}(5) = 5 + \text{sum_to_n}(4) = \dots$

Fill in the blank and run your code!



Great Job Today! See You Next Time!

Keep exploring the magic of recursion.
Keep practicing and have fun coding!

